

## Table of Content

|                                                                                                                      |           |
|----------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Question 1 .....</b>                                                                                              | <b>4</b>  |
| 1.1. Responses to Conceptual Challenges in Spatial Representation .....                                              | 4         |
| 1.2. Degeneracy in the Inverse Kinematics Solution .....                                                             | 5         |
| 1.3. Mathematical Ambiguity in Euler Angle Representation (Moving Frame $\mathbf{u}, \mathbf{v}, \mathbf{w}$ ) ..... | 5         |
| <b>Question 2 .....</b>                                                                                              | <b>7</b>  |
| 2.1. Computation of the Target Tissue Transformation ${}^E T_T$ .....                                                | 7         |
| 2.1.1 Extraction of Given Matrices .....                                                                             | 7         |
| 2.1.2. Construction of ${}^C T_T$ (TarTarget Relative to Camera).....                                                | 7         |
| 2.1.3. Efficient Computation of the Inverse ( ${}^E T_0$ ).....                                                      | 7         |
| 2.1.4. Applying the Chain of Frames Rule .....                                                                       | 8         |
| 2.2. Extracting the Equivalent Rotation Angle and the Rotation Axis Vector .....                                     | 8         |
| 2.2.1. Computing the Rotation Angle Using the Trace of the Matrix .....                                              | 8         |
| 2.2.2. Computing the Rotation Axis Vector .....                                                                      | 8         |
| <b>Question 3 .....</b>                                                                                              | <b>10</b> |
| 3.1. Preserve the Correct Trigonometric Quadrant by using <code>atan2</code> .....                                   | 10        |
| 3.2. Precise Identification of Computational Edges and Returning Solutions.....                                      | 10        |
| 3.3. Differentiating States and Reporting Angle Sums/Differences .....                                               | 11        |
| 3.4. Testing the Code Using the Section 2 array.....                                                                 | 12        |
| 3.5. Implementation .....                                                                                            | 13        |
| <b>Question 4 .....</b>                                                                                              | <b>16</b> |
| 4.1. Structural Analysis and Mathematical Equivalence of a Universal Joint.....                                      | 16        |
| 4.1.1 Proof of Modeling as Two Orthogonal Revolute Joints .....                                                      | 16        |
| 4.1.2. Conditions for Equivalence with the Roll–Pitch–Yaw Representation.....                                        | 16        |
| 4.2. Analysis of the Instantaneous Axis of Rotation and Workspace Constraints.....                                   | 16        |
| 4.2.1. Instantaneous Axis of Rotation (IAR) .....                                                                    | 16        |
| 4.2.2. Effect of a 10 Degrees Constraint on the Workspace .....                                                      | 17        |
| <b>Question 5 .....</b>                                                                                              | <b>18</b> |
| 5.1. Taylor Expansion and the Order at Which Non-Commutativity .....                                                 | 18        |
| 5.2. A realistic surgical scenario where “small angle assumption” Scenario .....                                     | 19        |
| 5.2.1. Position error of a point P on the tool tip.....                                                              | 20        |
| <b>Question 6 .....</b>                                                                                              | <b>21</b> |

|                                                                                                   |           |
|---------------------------------------------------------------------------------------------------|-----------|
| 6.1. Completion of the Homogeneous Transformation Matrix based on SO(3) Properties .....          | 21        |
| 6.1.1. Calculating the unknown $r_{32}$ .....                                                     | 21        |
| 6.1.2. Calculating the first column ( $u$ ) using the cross product.....                          | 21        |
| 6.1.2. Final Result .....                                                                         | 22        |
| 6.2. Efficient Computation of the Inverse Homogeneous Transformation Matrix ( ${}^1T_0$ ) .....   | 22        |
| 6.2.1. Extracting Components of ${}^0T_1$ .....                                                   | 22        |
| 6.2.2. Calculating the Transpose of the Rotation Matrix ( $R^T$ ) .....                           | 22        |
| 6.2.3. Calculating the New Translation Vector ( $-R^Tp$ ).....                                    | 22        |
| 6.2.4. Final ${}^1T_0$ Matrix Formation .....                                                     | 23        |
| 6.3. Correction and Validation of Homogeneous Transformation Matrices using SVD.....              | 23        |
| 6.3.1 General Code Description.....                                                               | 23        |
| 6.3.2. Mathematical Foundation of Correction .....                                                | 23        |
| 6.3.3 Final Output .....                                                                          | 23        |
| 6.3.4. Correction, Validation, and Verification of the Homogeneous Transformation Matrix<br>..... | 23        |
| <b>Question 7 .....</b>                                                                           | <b>26</b> |
| 7.1. Geometric Invariance Proof.....                                                              | 26        |
| 7.2. Tremor Calculation .....                                                                     | 26        |
| 7.2.1. Surgeon A .....                                                                            | 26        |
| 7.3 Incompetence Score Calculation.....                                                           | 27        |
| 7.3.1. Hand Calculation .....                                                                     | 28        |

## Question 1

### 1.1. Responses to Conceptual Challenges in Spatial Representation

Using the fixed-angle representation about the  $x, y, z$  axes (Roll–Pitch–Yaw), the overall rotation matrix is obtained as the product of three fundamental rotations about the fixed reference frame axes. The standard sequence is a rotation by  $\gamma$  (Yaw) about  $z$ , followed by  $\beta$  (Pitch) and finally  $\alpha$  (Roll) about  $x$ :

$$R_{rpy}(\alpha, \beta, \gamma) = R_z(\gamma) R_y(\beta) R_x(\alpha)$$

The elemental rotation matrices are defined as:

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix}$$

$$R_y(\beta) = \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix}$$

$$R_z(\gamma) = \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 \\ \sin \gamma & \cos \gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Mathematical analysis in the singular configuration, given  $\beta = \frac{\pi}{2}$  we have  $\cos(\frac{\pi}{2}) = 0$  and  $\sin(\frac{\pi}{2}) = 1$ . Substituting into  $R_y$ :

$$R_y\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$$

Computing the total rotation:

$$R_{rpy}\left(\alpha, \frac{\pi}{2}, \gamma\right) = R_z(\gamma) R_y\left(\frac{\pi}{2}\right) R_x(\alpha)$$

After matrix multiplication and applying trigonometric identities, the result simplifies to:

$$R_{rpy}\left(\alpha, \frac{\pi}{2}, \gamma\right) = \begin{bmatrix} 0 & \sin(\alpha - \gamma) & \cos(\alpha - \gamma) \\ 0 & \cos(\alpha - \gamma) & -\sin(\alpha - \gamma) \\ -1 & 0 & 0 \end{bmatrix}$$

From the final matrix, it is evident that  $\alpha$  and  $\gamma$  are no longer independent; only their difference  $(\alpha - \gamma)$  influences the orientation. Geometrically, when the Pitch angle reaches 90 degrees, the Roll axis aligns (collinearly but oppositely) with the Yaw axis. Consequently, rotations about these axes produce identical physical effects. This phenomenon is known as Gimbal Lock, representing the loss of one rotational degree of freedom.

## 1.2. Degeneracy in the Inverse Kinematics Solution

The inverse problem involves determining  $\alpha, \beta$  and  $\gamma$  from a given rotation matrix  $R = [r_{ij}]$ . Under normal conditions:

$$\alpha = \text{Atan2}(r_{32}, r_{33})$$

$$\beta = \text{Atan2}(-r_{31}, \sqrt{r_{11}^2 + r_{21}^2})$$

$$\gamma = \text{Atan2}(r_{21}, r_{11})$$

Degeneracy at  $\beta = \frac{\pi}{2}$ :

$$r_{11} = 0, r_{21} = 0, r_{31} = -1$$

Thus, both numerator and denominator in the  $\text{Atan2}$  expressions for  $\alpha$  and  $\gamma$  become zero, making them undefined. The standard extraction formulas fail due to singularity.

In this configuration, only the combination  $(\alpha - \gamma)$  can be determined. From a linear algebra perspective, the determinant of the analytical Jacobian becomes zero, implying infinitely many solutions, multiple  $(\alpha, \gamma)$  pairs correspond to the same physical orientation. Therefore, the mapping loses uniqueness, leading to degeneracy.

Therefore, the breakdown of the inverse problem precisely implies that the function mapping 'angles  $\rightarrow$  rotation matrix' is no longer one-to-one, and there exist infinitely many solutions for a single matrix.

## 1.3. Mathematical Ambiguity in Euler Angle Representation (Moving Frame

$u, v, w)$

If Euler angles with moving axes are used, singularities depend entirely on the chosen axis sequence.

- According to the equivalence theorem of rotation sequences

Extrinsic rotations about fixed axes  $x, y, z$  with angles  $(\alpha, \beta, \gamma)$  are mathematically equivalent to intrinsic rotations about moving axes  $x, y, z$  with angles  $(\alpha, \beta, \gamma)$ .

Thus, two cases arise:

1. Moving Sequence  $Z - Y - X$  (Tait-Bryan angles)

This produces the same matrix as before; hence, the singularity occurs in the same physical configuration and does not resolve the issue.

2. Proper Euler Sequence (e.g.,  $Z - Y - Z$ )

$$R_{zyz}(\phi, \theta, \psi) = R_z(\phi) R_y(\theta) R_z(\psi)$$

Singularity occurs when  $\sin(\theta) = 0$ ,  $\theta = 0$  or  $\pi$ .

From the previous physical configuration:

$$R(3,3) = 0 \Rightarrow \cos(\theta) = 0 \Rightarrow \theta = \frac{\pi}{2}$$

Since  $\theta = \frac{\pi}{2}$  does not satisfy the singularity condition, the system does not experience mathematical ambiguity in this representation. Changing parameterization effectively shifts the singularity to a different region of the robot's workspace.

## Question 2

### 2.1. Computation of the Target Tissue Transformation ${}^E T_T$

#### 2.1.1 Extraction of Given Matrices

According to the problem, the transformation matrices are:

$${}^0 T_C = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^0 T_E = \begin{bmatrix} 0.933 & -0.067 & -0.354 & 1 \\ -0.067 & 0.933 & -0.354 & 0 \\ 0.354 & 0.354 & 0.866 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

#### 2.1.2. Construction of ${}^C T_T$ (TarTarget Relative to Camera)

A translation of 0.5 along the  $z$  - axis and a rotation of  $\frac{\pi}{6}$  about the camera's  $y$  - axis yield:

$${}^C T_T = \begin{bmatrix} \cos(30^\circ) & 0 & \sin(30^\circ) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(30^\circ) & 0 & \cos(30^\circ) & 0.5 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.866 & 0 & 0.5 & 0 \\ 0 & 1 & 0 & 0 \\ -0.5 & 0 & 0.866 & 0.5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

#### 2.1.3. Efficient Computation of the Inverse ( ${}^E T_0$ )

To utilize the chain of frames, we require the transformation matrix of the end-effector relative to the reference frame, i.e.,  ${}^E T_0 = ({}^0 T_E)^{-1}$ .

The optimized method for inverting a homogeneous transformation matrix  $T = \begin{bmatrix} R & P \\ 0 & 1 \end{bmatrix}$  leverages the orthogonality of the rotation matrix  $T^{-1} = \begin{bmatrix} R^T & -R^T P \\ 0 & 1 \end{bmatrix}$ .

Given the matrix  $T_E^0$ :

$$R_E = \begin{bmatrix} 0.933 & -0.067 & 0.354 \\ 0.067 & 0.933 & 0.354 \\ -0.354 & -0.354 & 0.866 \end{bmatrix}, P_E = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

The transpose of the rotation matrix  $R_E^T$  is:

$$R_E^T = \begin{bmatrix} 0.933 & -0.067 & 0.354 \\ 0.067 & 0.933 & 0.354 \\ -0.354 & -0.354 & 0.866 \end{bmatrix}$$

Calculating the new translation vector:

$$-R_E^T P_E = - \begin{bmatrix} 0.933 & -0.067 & 0.354 \\ 0.067 & 0.933 & 0.354 \\ -0.354 & -0.354 & 0.866 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1.287 \\ -0.421 \\ -0.512 \end{bmatrix}$$

Thus, the inverse matrix is:

$${}^E T_0 = \begin{bmatrix} 0.933 & -0.067 & 0.354 & -1.287 \\ 0.067 & 0.933 & 0.354 & -0.421 \\ -0.354 & -0.354 & 0.866 & -0.512 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

#### 2.1.4. Applying the Chain of Frames Rule

To determine the position of the target relative to the surgical tool, we multiply the frames together:

$${}^E T_T = {}^E T_0 \cdot {}^0 T_C \cdot {}^C T_T$$

First, we compute  ${}^0 T_T = {}^0 T_C \cdot {}^C T_T$ :

$${}^0 T_T = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.866 & 0 & 0.5 & 0 \\ 0 & 1 & 0 & 0 \\ -0.5 & 0 & 0.866 & 0.5 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 0.866 & 0 & 0.5 & 1 \\ -0.5 & 0 & 0.866 & 1.5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Now, we perform the final multiplication  ${}^E T_T = {}^E T_0 \cdot {}^0 T_T$ :

$${}^E T_T = \begin{bmatrix} 0.933 & -0.067 & 0.354 & -1.287 \\ 0.067 & 0.933 & 0.354 & -0.421 \\ -0.354 & -0.354 & 0.866 & -0.512 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 & 0 & 0 \\ 0.866 & 0 & 0.5 & 1 \\ -0.5 & 0 & 0.866 & 1.5 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Upon performing the matrix multiplication (with entries rounded):

$${}^E T_T = \begin{bmatrix} -0.235 & -0.933 & 0.273 & -0.823 \\ 0.631 & -0.067 & 0.773 & 1.042 \\ -0.739 & 0.354 & 0.573 & -0.076 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

## 2.2. Extracting the Equivalent Rotation Angle and the Rotation Axis Vector

To align the surgical tool with the target tissue, the rotational component of the computed transformation matrix  ${}^E R_T$  must be analyzed:

$${}^T R_E = \begin{bmatrix} -0.235 & -0.933 & 0.273 \\ 0.631 & -0.067 & 0.773 \\ -0.739 & 0.354 & 0.573 \end{bmatrix}$$

### 2.2.1. Computing the Rotation Angle Using the Trace of the Matrix

For a rotation matrix, the sum of the diagonal elements is equal to  $1 + 2 \cos(\theta)$ :

$$\begin{aligned} \text{Tr}({}^E R_T) &= R_{11} + R_{22} + R_{33} = 0.271 \\ \Rightarrow 1 + 2 \cos(\theta) &= 0.271 \Rightarrow \cos(\theta) = -0.3645 \\ \theta &= 111.38^\circ \end{aligned}$$

### 2.2.2. Computing the Rotation Axis Vector

The eigenvector corresponding to the eigenvalue  $\lambda = 1$  defines the physical axis of rotation.

For  $\theta \neq 0, \pi$ , the equivalent closed-form formula is used:

$$v = \frac{1}{2 \sin(\theta)} \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix}$$

With  $\sin(111.38^\circ) \approx 0.931$ , the scale factor becomes:

$$\frac{1}{2 \sin(\theta)} \approx 0.537$$

Computing each term:

- $v_x = R_{32} - R_{23} = 0.354 - 0.773 = 0.419$
- $v_y = R_{13} - R_{31} = 0.273 - (-0.739) = 1.012$
- $v_z = R_{21} - R_{12} = 0.631 - (-0.933) = 1.564$

Thus, the normalized rotation axis is:

$$v = \begin{bmatrix} -0.225 \\ 0.543 \\ 0.84 \end{bmatrix}$$

To achieve full alignment with the target tissue, the tracking algorithm must rotate the surgical tool by  $\theta = 111.38^\circ$  around the physical axis defined by the unit vector.



## Question 3

### 3.1. Preserve the Correct Trigonometric Quadrant by using `atan2`

When extracting angles from inverse trigonometric functions, using operators such as `asin` or `acos` limits the resulting angle to a single half-plane (for example  $[-\frac{\pi}{2}, \frac{\pi}{2}]$ , for the inverse sine). This discards the sign information carried by both the sine and cosine terms, leading to ambiguous or incorrect reconstruction of the frame's orientation in 3D space.

The function `atan2(y, x)` resolves this issue by taking the numerator and denominator (related to the cosine term) separately, analyzing the sign of each, and assigning the resulting angle to the correct quadrant in the full range  $[-\pi, \pi]$ .

In the general case where  $\cos \beta \neq 0$ , two possible values exist for  $\cos \beta$ , a positive root `cb_pos` and a negative root `cb_neg`.

$$\beta = \text{atan2}(-r_{31}, \pm \sqrt{r_{11}^2 + r_{21}^2})$$

$$\alpha = \text{atan2}\left(\frac{r_{21}}{c\beta}, \frac{r_{11}}{c\beta}\right)$$

$$\gamma = \text{atan2}\left(\frac{r_{32}}{c\beta}, \frac{r_{33}}{c\beta}\right)$$

In the code, dividing the matrix terms by `cb_pos` or `cb_neg` constructs the appropriate  $(y, x)$  pairs for each `atan2` call. This guarantees that the computed roll ( $\alpha$ ) and yaw ( $\gamma$ ) angles are assigned to their correct quadrants, preserving the true spatial orientation of the frame.

```
import numpy as np
import math

def extract_rpy_normal(R):
    """
    Robust Algorithm for Converting a Rotation Matrix to Roll-Pitch-Yaw Angles
    Rotation Sequence: Rz(gamma) * Ry(beta) * Rx(alpha)

    Logic Flow:
    1. Extract Beta using r31.
    2. Check for singularity condition (Gimbal Lock) using epsilon tolerance.
    3. In case of singularity: Resolve ambiguity using linearized relationships.
    4. In normal state: Unique extraction of all angles using atan2.
    """
    # Calculate the two possible roots for cosine beta (positive and negative)
    cb_pos = math.sqrt(R[0, 0]**2 + R[1, 0]**2)
    cb_neg = -cb_pos
```

```

# First solution
beta1 = math.atan2(-R[2, 0], cb_pos)
alpha1 = math.atan2(R[1, 0] / cb_pos, R[0, 0] / cb_pos)
gamma1 = math.atan2(R[2, 1] / cb_pos, R[2, 2] / cb_pos)

# Second solution
beta2 = math.atan2(-R[2, 0], cb_neg)
alpha2 = math.atan2(R[1, 0] / cb_neg, R[0, 0] / cb_neg)
gamma2 = math.atan2(R[2, 1] / cb_neg, R[2, 2] / cb_neg)

return [(gamma1, beta1, alpha1), (gamma2, beta2, alpha2)]

```

### 3.2. Precise Identification of Computational Edges and Returning Solutions

A computational edge, or singularity, occurs when  $\cos \beta$ . Since the matrix element  $r_{31} = -\sin \beta$  the condition  $\cos \beta = 0$  is equivalent to  $\sin \beta = \pm 1$ , or  $|r_{31}| = 1$ . In such scenarios, standard algorithms encounter division-by-zero errors because  $\cos \beta$  appears in the denominator of the equations for  $\alpha$  and  $\gamma$ .

To circumvent numerical errors (Floating Point Errors), these computational edges are detected using a tolerance ( $\text{tol} = 1\text{e-}6$ ):

1. Detection of  $\beta = \frac{\pi}{2}$ : This occurs when  $r_{31} \approx -1$  (i.e.,  $\text{abs}(R[2, 0] + 1.0) < \text{tol}$ ).
2. Detection of  $\beta = -\frac{\pi}{2}$ : This occurs when  $r_{31} \approx 1$  (i.e.,  $\text{abs}(R[2, 0] - 1.0) < \text{tol}$ ).

If neither of these conditions is met, the function `extract_rpy_normal` is invoked. This function returns both possible solutions that arise in the general case one assuming  $\cos \beta > 0$  and the other assuming  $\cos \beta < 0$  packaged within a list.

```

def rotation_matrix_to_rpy(R):
    tol = 1e-6
    if abs(R[2, 0] + 1.0) < tol:
        return extract_rpy_edge_cases(R, is_positive_pi2=True)
    elif abs(R[2, 0] - 1.0) < tol:
        return extract_rpy_edge_cases(R, is_positive_pi2=False)
    else:
        return extract_rpy_normal(R)

```

### 3.3. Differentiating States and Reporting Angle Sums/Differences

In the singular state  $\cos \beta = 0$ , the Roll and Yaw axes align (Gimbal Lock), simplifying the rotation matrix. At this point,  $\alpha$  and  $\gamma$  cannot be determined independently; only a linear combination of them can be computed.

1. State  $\beta = \frac{\pi}{2}$  ( $\sin \beta = 1$ )

In this specific case, the matrix equations simplify to:

$$r_{22} = \cos(\alpha - \gamma), r_{23} = \sin(\alpha - \gamma)$$

The expression `math.atan2(R[1, 2], R[1, 1])` precisely computes  $\alpha - \gamma$ . To yield a unique solution from the infinite possibilities, the algorithm assumes  $\gamma$ , thus setting  $\alpha$  equal to this computed difference.

2. State  $\beta = -\frac{\pi}{2}$  ( $\sin \beta = -1$ )

Under this condition, the matrix equations simplify differently:

$$r_{22} = \cos(\alpha + \gamma), r_{12} = -\sin(\alpha + \gamma)$$

Therefore, the sum of the angles is calculable:

$$\alpha + \gamma = \text{atan2}(-r_{12}, r_{22})$$

The expression `math.atan2(-R[0, 1], R[1, 1])` accurately extracts the sum  $\alpha + \gamma$ .

Again, by assuming  $\gamma = 0$  to obtain a single result, the value of  $\alpha$  is determined.

In both edge-case scenarios, the algorithm correctly distinguishes the state and returns two identical tuples. This is done to maintain a consistent output structure, providing a unique solution under the assumption of  $\gamma = 0$ .

```
def extract_rpy_edge_cases(R, is_positive_pi2):
    # Under singular conditions, infinite solutions exist; we assume one angle is
    # zero.
    gamma = 0.0
    if is_positive_pi2:
        beta = math.pi / 2.0
        alpha = math.atan2(R[1, 2], R[1, 1]) + gamma
        print("Edge Case: beta = +pi/2")
    else:
        beta = -math.pi / 2.0
        alpha = math.atan2(-R[0, 1], R[1, 1]) - gamma
        print("Edge Case: beta = -pi/2")
    return [(gamma, beta, alpha), (gamma, beta, alpha)]
```

### 3.4. Testing the Code Using the [Section 2](#) array

The rotation matrix provided as input to the code `R_part2` is the same matrix derived in [Part Two](#). In this matrix, the element  $r_{31}$  is equal to  $-0.739$ .

Mathematically, since the condition  $|R_{31}| = 1$  is not met ( $|-0.739| \neq 1$ ), the algorithm correctly determines that there is no singularity (Gimbal Lock), and the frame is not at the computational edge. Therefore, the `extract_rpy_normal` function is called.

In Euler angles (and Roll-Pitch-Yaw), there are always two distinct sets of angles that result in precisely the same final orientation in 3D space. One solution typically places the Pitch angle  $\beta$  within the range  $[-\frac{\pi}{2}, \frac{\pi}{2}]$ , while the other solution reaches the same position by “flipping” the frame (Pitch greater than 90 degrees or less than -90 degrees) and proportionally adjusting the Roll and Yaw.

```
# Test Code for the Matrix Solved in Part Two
# =====

if __name__ == "__main__":
    # The rotation matrix extracted from Part Two (computational)
```

```

R_part2 = np.array([
    [-0.235, -0.933,  0.273],
    [ 0.631, -0.067,  0.773],
    [-0.739,  0.354,  0.573]
])

print("Testing the Rotation Matrix from Part Two")
solutions = rotation_matrix_to_rpy(R_part2)

for i, sol in enumerate(solutions):
    gamma_rad, beta_rad, alpha_rad = sol
    # Convert radians to degrees for better readability
    gamma_deg = math.degrees(gamma_rad)
    beta_deg = math.degrees(beta_rad)
    alpha_deg = math.degrees(alpha_rad)

```

After executing the code, the output is obtained, as shown in Figure 3.1.

```

Testing the Rotation Matrix from Part Two

Solution 1:
  Radians -> Roll (gamma):  0.5534 | Pitch (beta):  0.8319 | Yaw (alpha):  1.9273
  Degrees -> Roll (gamma):  31.71° | Pitch (beta):  47.66° | Yaw (alpha): 110.43°

Solution 2:
  Radians -> Roll (gamma): -2.5882 | Pitch (beta):  2.3097 | Yaw (alpha): -1.2143
  Degrees -> Roll (gamma): -148.29° | Pitch (beta): 132.34° | Yaw (alpha): -69.57°

```

Figure 3.1. Motion degrees for testing matrix

Solution 1 shows the Pitch angle is  $47.66^\circ$ . Since this angle falls within the acute range (between  $-90^\circ$  and  $90^\circ$ ), this solution is generally considered the “natural” or standard configuration of the device that has a shorter path to the target.

Solution 2 shows the Pitch is  $132.34^\circ$ . This is an alternative configuration. Physically, if the tool first undergoes different Roll and Yaw movements (with a change in sign and value compared to Solution 1) and then rotates its Pitch by the reflex angle, it will arrive at the exact same final position as Solution 1.

As can be observed, since this matrix does not encounter a singularity, the algorithm successfully enters the non-singular block (normal state) and extracts all three angles independently and accurately while preserving the correct quadrant of the trigonometric circle.

### 3.5. Implementation

```

import numpy as np
import math

def extract_rpy_normal(R):
    """

```

Robust Algorithm for Converting a Rotation Matrix to Roll-Pitch-Yaw Angles  
 Rotation Sequence:  $R_z(\gamma) * R_y(\beta) * R_x(\alpha)$

Logic Flow:

1. Extract Beta using  $r_{31}$ .
2. Check for singularity condition (Gimbal Lock) using epsilon tolerance.
3. In case of singularity: Resolve ambiguity using linearized relationships (sum or difference of angles).
4. In normal state: Unique extraction of all angles using  $\text{atan2}$ .

"""

*# Calculate the two possible roots for cosine beta (positive and negative)*

`cb_pos = math.sqrt(R[0, 0]**2 + R[1, 0]**2)`

`cb_neg = -cb_pos`

*# First solution*

`beta1 = math.atan2(-R[2, 0], cb_pos)`

`alpha1 = math.atan2(R[1, 0] / cb_pos, R[0, 0] / cb_pos)`

`gamma1 = math.atan2(R[2, 1] / cb_pos, R[2, 2] / cb_pos)`

*# Second solution*

`beta2 = math.atan2(-R[2, 0], cb_neg)`

`alpha2 = math.atan2(R[1, 0] / cb_neg, R[0, 0] / cb_neg)`

`gamma2 = math.atan2(R[2, 1] / cb_neg, R[2, 2] / cb_neg)`

`return [(gamma1, beta1, alpha1), (gamma2, beta2, alpha2)]`

**def** `extract_rpy_edge_cases(R, is_positive_pi2):`

*# Under singular conditions, infinite solutions exist; we assume one angle is zero.*

`gamma = 0.0`

**if** `is_positive_pi2:`

`beta = math.pi / 2.0`

`alpha = math.atan2(R[1, 2], R[1, 1]) + gamma`

`print("Edge Case: beta = +pi/2")`

**else:**

`beta = -math.pi / 2.0`

`alpha = math.atan2(-R[0, 1], R[1, 1]) - gamma`

`print("Edge Case: beta = -pi/2")`

`return [(gamma, beta, alpha), (gamma, beta, alpha)]`

**def** `rotation_matrix_to_rpy(R):`

`tol = 1e-6`

**if** `abs(R[2, 0] + 1.0) < tol:`

`return extract_rpy_edge_cases(R, is_positive_pi2=True)`

**elif** `abs(R[2, 0] - 1.0) < tol:`

```
        return extract_rpy_edge_cases(R, is_positive_pi2=False)
    else:
        return extract_rpy_normal(R)

# =====
# Test Code for the Matrix Solved in Part Two
# =====
if __name__ == "__main__":
    # The rotation matrix extracted from Part Two (computational)
    R_part2 = np.array([
        [-0.235, -0.933, 0.273],
        [ 0.631, -0.067, 0.773],
        [-0.739, 0.354, 0.573]
    ])

    print("Testing the Rotation Matrix from Part Two")
    solutions = rotation_matrix_to_rpy(R_part2)

    for i, sol in enumerate(solutions):
        gamma_rad, beta_rad, alpha_rad = sol
        # Convert radians to degrees for better readability
        gamma_deg = math.degrees(gamma_rad)
        beta_deg = math.degrees(beta_rad)
        alpha_deg = math.degrees(alpha_rad)

        print(f"\nSolution {i+1}:")
        print(f"  Radians -> Roll (gamma): {gamma_rad:8.4f} | Pitch (beta): {beta_rad:8.4f} | Yaw (alpha): {alpha_rad:8.4f}")
        print(f"  Degrees -> Roll (gamma): {gamma_deg:8.2f}° | Pitch (beta): {beta_deg:8.2f}° | Yaw (alpha): {alpha_deg:8.2f}°")
```

## Question 4

### 4.1. Structural Analysis and Mathematical Equivalence of a Universal Joint

A Universal joint in robotics is designed to transmit rotational motion between two intersecting links.

#### 4.1.1 Proof of Modeling as Two Orthogonal Revolute Joints

Assume that the coordinate frame {5} (link 5) serves as the base frame of the wrist. A Universal joint can be modeled as two consecutive rotations:

1. First rotation ( $\theta_1$ ): about the axis  $\hat{z}_5$ , generating an intermediate frame.
2. Second rotation ( $\theta_2$ ): about the new  $\hat{y}$ -axis of the intermediate frame, which due to the cross-shaped mechanism remains perpendicular to the first axis.

The total rotation matrix is therefore:

$$R_{joint} = R_z(\theta_1) R_y(\theta_2)$$

$$R_{joint} = \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 \\ \sin \theta_1 & \cos \theta_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos \theta_2 & 0 & \sin \theta_2 \\ 0 & 1 & 0 \\ -\sin \theta_2 & 0 & \cos \theta_2 \end{bmatrix}$$

$$R_{joint} = \begin{bmatrix} \cos \theta_1 \cos \theta_2 & -\sin \theta_1 & \cos \theta_1 \sin \theta_2 \\ \sin \theta_1 \cos \theta_2 & \cos \theta_1 & \sin \theta_1 \sin \theta_2 \\ -\sin \theta_2 & 0 & \cos \theta_2 \end{bmatrix}$$

#### 4.1.2. Conditions for Equivalence with the Roll–Pitch–Yaw Representation

The fixed-angle  $x - y - z$  rotation matrix (RPY), discussed previously, is given by:

$$R_{rpy} = R_z(\gamma) R_y(\beta) R_x(\alpha)$$

By comparing  $R_{joint}$  and  $R_{rpy}$ , it is evident that the two matrices become identical only when  $\alpha = 0$ .

In this configuration, the surgical tool exhibits no rotation about its longitudinal axis  $x_e$ .

A Universal joint can only control the pointing direction of the tool in space (Pitch and Yaw), but cannot induce rotation about its own axis (Roll). Therefore, it represents a 2-degree-of-freedom subset of the full 3-DOF rotational space.

## 4.2. Analysis of the Instantaneous Axis of Rotation and Workspace Constraints

This section is directly related to your lecture slides and the concept of eigenvalues.

### 4.2.1. Instantaneous Axis of Rotation (IAR)

According to Euler's rotation theorem, any rotation in three-dimensional space can be described by a single fixed axis. As indicated in your slides, for any rotation matrix  $R$ , the eigenvalues are  $1, e^{i\theta}, e^{-i\theta}$

- Role of the eigenvector, if  $v$  is the eigenvector corresponding to the eigenvalue 1, then  $Rv = v$ . This implies that  $v$  remains unchanged under the rotation, and therefore represents the axis of rotation.

In a Universal joint, the matrix  $R_{joint}$  depends on two independent variables  $\theta_1$  and  $\theta_2$ . As these angles vary during operation, the entries of the matrix change, and consequently the eigenvector corresponding to  $\lambda = 1$  also changes in space.

The instantaneous axis of rotation is not fixed, but continuously sweeps through three-dimensional space as the joint moves.

#### 4.2.2. Effect of a 10 Degrees Constraint on the Workspace

The constraint  $\beta, \gamma \leq 10^\circ$  implies that the end-effector cannot move over a full spherical region.

The resulting workspace is a spherical cone, with its apex located at the center of the Universal joint.

Let  $L$  denote the length of the tool (from wrist to tip). The occupied volume in 3D space, using spherical coordinates, is:

$$V = \frac{2\pi L^3}{3} \int_0^{10^\circ} \sin \phi \, d\phi = \frac{2\pi L^3}{3} [-\cos \phi]_0^{10^\circ}$$

$$V = \frac{2\pi L^3}{3} (1 - \cos 10^\circ)$$

$$V \approx 0.031 L^3$$

This corresponds to only about 1.5% of a full hemisphere's volume.

Such a severe limitation implies that for larger maneuvers, the robot must rely on upstream joints (the arm), which increases the likelihood of collision with the constrained surgical environment.



## Question 5

### 5.1. Taylor Expansion and the Order at Which Non-Commutativity

We consider two very small rotations:

- First rotation  
 $\delta\theta_1$  about the fixed x-axis  $\Rightarrow$  rotation matrix  $R_x(\delta\theta_1)$
- Second rotation  
 $\delta\theta_2$  about the fixed y-axis  $\Rightarrow$  rotation matrix  $R_y(\delta\theta_2)$

For very small angles:

$$\sin \delta\theta \approx \delta\theta, \cos \delta\theta \approx 1 - \frac{1}{2}\delta\theta^2$$

Rotation around x:

$$R_x(\delta\theta_1) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 - \frac{\delta\theta_1^2}{2} & -\delta\theta_1 \\ 0 & \delta\theta_1 & 1 - \frac{\delta\theta_1^2}{2} \end{bmatrix}$$

Rotation around y:

$$R_y(\delta\theta_2) = \begin{bmatrix} 1 - \frac{\delta\theta_2^2}{2} & 0 & \delta\theta_2 \\ 0 & 1 & 0 \\ -\delta\theta_2 & 0 & 1 - \frac{\delta\theta_2^2}{2} \end{bmatrix}$$

#### 5.1.1. When do the Rotations Commute

We compare:

$$R_y(\delta\theta_2)R_x(\delta\theta_1) \text{ vs } R_x(\delta\theta_1)R_y(\delta\theta_2)$$

Keeping only first-order terms (dropping all  $\delta\theta^2$ ), Approximate:

$$R_x \approx I + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -\delta\theta_1 \\ 0 & \delta\theta_1 & 0 \end{bmatrix}$$

$$R_y \approx I + \begin{bmatrix} 0 & 0 & \delta\theta_2 \\ 0 & 0 & 0 \\ -\delta\theta_2 & 0 & 0 \end{bmatrix}$$

Then:

$$R_y R_x \approx I + A_y + A_x$$

$$R_x R_y \approx I + A_x + A_y$$

They are identical at first order. Non-commutativity disappears at first order.

The first term where it appears is:

$$O(\delta\theta_1 \delta\theta_2)$$

So:

$$[R_x, R_y] = O(\delta\theta_1 \delta\theta_2)$$

Rotations become commutative only when all second-order terms are neglected. Non-commutativity starts at second order.

## 5.2. A realistic surgical scenario where “small angle assumption” Scenario

A robotic surgical tool operates inside a very tight cavity.

The surgeon applies two extremely small joystick motions intended to produce a pure rotation.

The controller simplifies the kinematics using:

- small-angle approximations
- dropping all second-order terms

But in reality:

- the non-commutativity error  $\sim \delta\theta_1 \delta\theta_2$
- produces a linear displacement at the tool tip, magnified by its length.

Example with an end-effector length  $L = 30$  cm:

$$\delta\theta_1 = 1^\circ = 0.017 \text{ rad}$$

$$\delta\theta_2 = 1^\circ = 0.017 \text{ rad}$$

Position error:

$$\Delta x \approx L(\delta\theta_1 \delta\theta_2) = 0.30 \times (0.017 \times 0.017) \approx 0.087 \text{ mm}$$

0.1 mm of unintended motion is enough to cause contact with critical tissue (vessels, ducts, nerves).

Thus, relying on “small angle  $\Rightarrow$  commutativity” can be dangerous in surgical robotics.

### 5.2.1. Position error of a point P on the tool tip

Two transformation paths exist:

Path 1:

$${}^0T_1 \quad {}^1T_2$$

Path 2:

$${}^0T_2 \quad {}^2T_1$$

Let point  $P$  be expressed in frame E:

$${}^EP$$

Position of P through each path:

$${}^0P_1 = {}^0T_1 \quad {}^1T_2 \quad {}^EP$$

$${}^0P_2 = {}^0T_2 \quad {}^2T_1 \quad {}^EP$$

Position error vector:

$$\Delta P = {}^0P_1 - {}^0P_2$$

Magnitude (norm) of the position error:

$$\|\Delta P\| = \| ({}^0T_1 \quad {}^1T_2 - {}^0T_2 \quad {}^2T_1) \quad {}^EP \|$$

This expression directly captures the physical error caused by non-commutativity of rotation matrices.

## Question 6

### 6.1. Completion of the Homogeneous Transformation Matrix based on SO(3) Properties

Based on the available data, the homogeneous transformation matrix  ${}^0T_1$  is defined as follows, containing several unknowns in the rotation sub-matrix:

$${}^0T_1 = \begin{bmatrix} r_{11} & 0.067 & -0.354 & -1.1 \\ r_{21} & 0.933 & -0.354 & 1.0 \\ r_{31} & r_{32} & 0.866 & 1.1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

For this matrix to be a valid transformation matrix in 3D space, its rotation part (*the upper – left  $3 \times 3$  sub – matrix*) must belong to the Special Orthogonal group  $SO(3)$ . This implies that its columns (vectors  $u$ ,  $v$ , and  $w$ ) must be orthonormal and follow the right-hand rule.

#### 6.1.1. Calculating the unknown $r_{32}$

According to the orthogonality constraint, the dot product of the second and third columns ( $v$  and  $w$ ) must be zero ( $v \cdot w = 0$ ):

$$v = \begin{bmatrix} 0.067 \\ 0.933 \\ r_{32} \end{bmatrix}, w = \begin{bmatrix} -0.354 \\ -0.354 \\ 0.866 \end{bmatrix}$$

$$(0.067 \times -0.354) + (0.933 \times -0.354) + (r_{32} \times 0.866) = 0$$

$$-0.354(0.067 + 0.933) + 0.866r_{32} = 0$$

$$-0.354(1) + 0.866r_{32} = 0 \implies r_{32} = \frac{0.354}{0.866} \approx 0.4088$$

#### 6.1.2. Calculating the first column ( $u$ ) using the cross product

To satisfy the right-hand rule, the first column must be the cross product of the second and third columns ( $u = v \times w$ ):

$$u = \begin{bmatrix} r_{11} \\ r_{21} \\ r_{31} \end{bmatrix} = \begin{bmatrix} 0.067 \\ 0.933 \\ 0.4088 \end{bmatrix} \times \begin{bmatrix} -0.354 \\ -0.354 \\ 0.866 \end{bmatrix}$$

The component calculations are as follows:

$$r_{11} = (0.933 \times 0.866) - (0.4088 \times -0.354) \approx 0.9527$$

$$r_{21} = (0.4088 \times -0.354) - (0.067 \times 0.866) \approx -0.2027$$

$$r_{31} = (0.067 \times -0.354) - (0.933 \times -0.354) \approx 0.3066$$

### 6.1.2. Final Result

The calculated unknowns to complete the matrix are:

$$r_{11} = 0.9527, \quad r_{21} = -0.2027, \quad r_{31} = 0.3066, \quad r_{32} = 0.4088$$

## 6.2. Efficient Computation of the Inverse Homogeneous Transformation Matrix ( ${}^1T_0$ )

In this section, the objective is to compute the inverse of the homogeneous transformation matrix  ${}^0T_1$ . Utilizing the structural properties of such matrices, the inverse is calculated efficiently rather than using general inversion methods:

$${}^1T_0 = ({}^0T_1)^{-1} = \begin{bmatrix} R^T & -R^T p & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

### 6.2.1. Extracting Components of ${}^0T_1$

Based on the previously determined values, the transformation matrix is defined as:

$${}^0T_1 = \begin{bmatrix} 0.9527 & 0.067 & -0.354 & -1.1 \\ -0.2027 & 0.933 & -0.354 & 1.0 \\ 0.3066 & 0.4088 & 0.866 & 1.1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Where the rotation matrix ( $R$ ) and translation vector ( $p$ ) are:

$$R = \begin{bmatrix} 0.9527 & 0.067 & -0.354 \\ -0.2027 & 0.933 & -0.354 \\ 0.3066 & 0.4088 & 0.866 \end{bmatrix}, p = \begin{bmatrix} -1.1 \\ 1.0 \\ 1.1 \end{bmatrix}$$

### 6.2.2. Calculating the Transpose of the Rotation Matrix ( $R^T$ )

$$R^T = \begin{bmatrix} 0.9527 & -0.2027 & 0.3066 \\ 0.067 & 0.933 & 0.4088 \\ -0.354 & -0.354 & 0.866 \end{bmatrix}$$

### 6.2.3. Calculating the New Translation Vector ( $-R^T p$ )

The components of the inverse translation vector are calculated as follows:

$$p' = -R^T p = - \begin{bmatrix} 0.9527 & -0.2027 & 0.3066 \\ 0.067 & 0.933 & 0.4088 \\ -0.354 & -0.354 & 0.866 \end{bmatrix} \begin{bmatrix} -1.1 \\ 1.0 \\ 1.1 \end{bmatrix}$$

$$x' = -[(0.9527 \times -1.1) + (-0.2027 \times 1.0) + (0.3066 \times 1.1)] \approx 0.9134$$

$$y' = -[(0.067 \times -1.1) + (0.933 \times 1.0) + (0.4088 \times 1.1)] \approx -1.3090$$

$$z' = -[(-0.354 \times -1.1) + (-0.354 \times 1.0) + (0.866 \times 1.1)] \approx -0.9880$$

#### 6.2.4. Final ${}^1T_0$ Matrix Formation

By substituting the above values, the final inverse matrix is obtained:

$${}^1T_0 = \begin{bmatrix} 0.9527 & -0.2027 & 0.3066 & 0.9134 \\ 0.067 & 0.933 & 0.4088 & -1.3090 \\ -0.354 & -0.354 & 0.866 & -0.9880 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

### 6.3. Correction and Validation of Homogeneous Transformation Matrices using SVD

This section focuses on rectifying noisy transformation matrices and ensuring their geometric integrity. Mathematically, the rotation component of a transformation matrix must remain orthogonal, and its determinant must strictly equal 1 ( $\det(R) = 1$ ).

#### 6.3.1 General Code Description

The provided code performs the following tasks systematically:

- **Component Extraction:** Separates the rotation matrix ( $R_{3 \times 3}$ ) and the translation vector ( $P_{3 \times 1}$ ) from the input  $4 \times 4$  matrix.
- **Reflection Check:** Computes the determinant of the rotation part; a negative value indicates a reflection error, which is geometrically invalid for pure rotation.
- **SVD Optimization:** Utilizes Singular Value Decomposition to find the nearest orthogonal matrix to the noisy rotation part using the least-squares method.
- **Matrix Reconstruction:** Reassembles the corrected components into a standard  $4 \times 4$  homogeneous transformation matrix.

#### 6.3.2. Mathematical Foundation of Correction

To find the closest valid rotation matrix ( $R_{valid}$ ), the following algorithm is implemented:

$$R_{noisy} = U\Sigma V^T$$

$$R_{valid} = UV^T$$

This approach guarantees that the resulting matrix satisfies the orthogonality condition  $R^T R = I$ .

#### 6.3.3 Final Output

After applying the `fix_transformation_matrix` function to a distorted matrix, the determinant is restored to exactly 1, making the matrix suitable for kinematic calculations:

$$\det(R_{valid}) = 1.0$$

#### 6.3.4. Correction, Validation, and Verification of the Homogeneous Transformation Matrix

In this section, a homogeneous transformation matrix containing numerical noise in its rotation component was corrected and validated using the designed function.

The noisy input matrix used for testing is defined as:

$$T_{test} = \begin{bmatrix} 0.9 & 0.1 & -0.4 & -1.1 \\ -0.2 & 0.9 & -0.4 & 1.0 \\ 0.3 & 0.4 & 0.9 & 1.1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

After applying the SVD optimization process, the corrected matrix was obtained (Figure 6.1):

```
Fixed Matrix:
[[ 0.9253  0.0795 -0.3707 -1.1  ]
 [-0.2224  0.9058 -0.3608  1.   ]
 [ 0.3071  0.4163  0.8558  1.1  ]
 [ 0.      0.      0.      1.   ]]

Determinant of the new rotation part: 1.0
```

Figure 6.1. Fixed Matrix

The primary validation metric for rotation matrices is the determinant of the 3x3 component. As indicated in the Figure 6.1. Determinant of the new rotation part, 1.0. This result confirms that the matrix now represents a pure rotation (without reflection or scale distortion) and is fully valid for kinematic calculations.

```
import numpy as np

def fix_transformation_matrix(T_noisy):
    """
    Fits the closest valid homogeneous transformation matrix to a noisy input
    matrix.
    """
    # Extract rotation part (R) and translation part (P)
    R_noisy = T_noisy[0:3, 0:3]
    P = T_noisy[0:3, 3]

    # Check initial determinant for reflection errors
    det_initial = np.linalg.det(R_noisy)
    if det_initial < 0:
        raise ValueError(f"Reflection Error: Rotation part determinant is
        negative (det={det_initial:.4f}). Reconstruction is not possible.")

    # Apply SVD to find the nearest orthogonal matrix (Least Squares fit)
    U, S, Vt = np.linalg.svd(R_noisy)
    R_valid = np.dot(U, Vt)

    # Final check to ensure the orthogonal matrix determinant is exactly 1 (and
    not -1)
```

```
    if np.linalg.det(R_valid) < 0:
        raise ValueError("Reflection Error: The fitted matrix contains a mirror
reflection.")

    # Construct the corrected homogeneous transformation matrix
    T_valid = np.eye(4)
    T_valid[0:3, 0:3] = R_valid
    T_valid[0:3, 3] = P

    return T_valid

# --- Example Usage ---
# A hypothetical noisy matrix with positive determinant but requiring correction
T_test = np.array([
    [ 0.9,  0.1, -0.4, -1.1],
    [-0.2,  0.9, -0.4,  1.0],
    [ 0.3,  0.4,  0.9,  1.1],
    [ 0.0,  0.0,  0.0,  1.0]
])

try:
    T_fixed = fix_transformation_matrix(T_test)
    print("Fixed Matrix:\n", np.round(T_fixed, 4))
    print("\nDeterminant of the new rotation part:",
round(np.linalg.det(T_fixed[0:3, 0:3]), 4))
except ValueError as e:
    print(e)
```



## Question 7

### 7.1. Geometric Invariance Proof

In the assessment of motor skills, such as detecting involuntary surgical hand tremor, it is necessary to extract the local rotation of the tool independently of its global trajectory. Let  $\{W\}$  be the global reference (World) coordinate frame and  $\{T\}$  be the coordinate frame attached to the tool (Tool). The relative rotation matrix of the tool between time  $t$  and  $t + \Delta t$ , relative to its own local frame at time  $t$ , is defined as:

$$R_{rel} = ({}^W R_{T(t)})^T \cdot {}^W R_{T(t+\Delta t)}$$

To prove that this metric is geometrically invariant (i.e., independent of the global reference frame or camera position), suppose we have a new observer frame named  $\{C\}$  whose rotation relative to the global reference is denoted by  ${}^C R_W$ .

The tool's rotation in this new frame  $\{C\}$  at two different times will be:

$$\begin{aligned} {}^C R_{T(t)} &= {}^C R_W \cdot {}^W R_{T(t)} \\ {}^C R_{T(t+\Delta t)} &= {}^C R_W \cdot {}^W R_{T(t+\Delta t)} \end{aligned}$$

Substituting these new equations into the relative rotation formula yields:

$$\begin{aligned} R'_{rel} &= ({}^C R_{T(t)})^T \cdot {}^C R_{T(t)} = ({}^C R_W \cdot {}^W R_{T(t)})^T \cdot {}^C R_W \cdot {}^W R_{T(t+\Delta t)} \\ R'_{rel} &= ({}^W R_{T(t)})^T \cdot ({}^C R_W)^T \cdot {}^C R_W \cdot {}^W R_{T(t+\Delta t)} = ({}^W R_{T(t)})^T \cdot I \cdot {}^W R_{T(t+\Delta t)} = ({}^W R_{T(t)})^T \cdot {}^W R_{T(t+\Delta t)} \\ R'_{rel} &= R_{rel} \end{aligned}$$

As proven, the relative rotation matrix yields the exact same value regardless of the observer frame. This demonstrates that the angular change of the tool is an intrinsic property, and changing the viewing angle has no effect on the measurement of the tool's internal rotations.

### 7.2. Tremor Calculation

To evaluate the smoothness of motion for each surgeon, the relative rotation matrix  $R_{rel}$  and the corresponding net rotation angle  $\Delta\theta$  (in degrees) are computed. A smaller rotation angle indicates less parasitic motion and smoother instrument control.

#### 7.2.1. Surgeon A

Given:

$$R_A(t) = I_{3 \times 3}, R_A(t + \Delta t) = \begin{bmatrix} 0.9998 & -0.0175 & 0 \\ 0.0175 & 0.9998 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The relative rotation matrix is:

$$R_{rel,A} = R_A(t)^T R_A(t + \Delta t) = R_A(t + \Delta t)$$

The rotation angle is derived from the trace:

$$\theta_A = \cos^{-1} \left( \frac{\text{tr}(R_{rel,A}) - 1}{2} \right)$$

$$\text{tr}(R_{rel,A}) = 0.9998 + 0.9998 + 1 = 2.9996$$

$$\frac{2.9996 - 1}{2} = \frac{1.9996}{2} = 0.9998$$

$$\theta_A = \cos^{-1}(0.9998) = 1.146^\circ$$

### 7.2.2. Surgeon B

Given:

$$R_B(t) = I_{3 \times 3}, R_B(t + \Delta t) = \begin{bmatrix} 0.9962 & -0.0174 & 0.0858 \\ 0.0175 & 0.9998 & 0 \\ -0.0858 & 0.0015 & 0.9963 \end{bmatrix}$$

The relative rotation matrix is:

$$R_{rel,B} = R_B(t)^T R_B(t + \Delta t) = R_B(t + \Delta t)$$

Trace calculation:

$$\text{tr}(R_{rel,B}) = 0.9962 + 0.9998 + 0.9963 = 2.9923$$

$$\frac{2.9923 - 1}{2} = \frac{1.9923}{2} = 0.99615$$

$$\theta_B = \cos^{-1}(0.99615) = 5.029^\circ$$

Since  $\theta_A < \theta_B$ , Surgeon A exhibits smoother motion (less rotational tremor) compared to Surgeon B.

## 7.3 Incompetence Score Calculation

A Python function is developed to compute an incompetence score – a kinematic metric that quantifies surgical skill based on the sum of squared angular changes between consecutive rotation matrices along a tool path. A lower score indicates smoother motion (less parasitic rotation) and greater surgical expertise.

```
def incompetence_score(transforms):
    s = 0

    R_t = None
    for R_deltat in transforms:
        if R_t is None:
            R_t = R_deltat
            continue

        Rrel = R_t.T @ R_deltat
```

```

R_t = R_deltat
cos_theta = np.clip((np.trace(Rrel) - 1) / 2, -1.0, 1.0)
theta = np.arccos(cos_theta)
theta = np.rad2deg(theta)
s += theta**2

return s

```

The function iterates through a sequence of rotation matrices, computing the relative rotation between consecutive frames using  $R_{rel} = R_t.T @ R_{deltat}$ . This relative rotation is invariant to the choice of reference coordinate system. For each relative rotation, the net angle is extracted using the formula  $\theta = \arccos((\text{tr}(R_{rel}) - 1) / 2)$ . The `np.clip` function ensures numerical stability by clamping the argument of `arccos` to the valid range  $[-1, 1]$ , preventing errors caused by floating-point imprecision. The angle is converted to degrees, squared (to heavily penalize large parasitic motions), and accumulated into the final score.

```

        [0.0175, 0.9998, 0],
        [0,0,1]])

B = np.array([[0.9962, -0.0174, 0.0858],
              [0.0175, 0.9998, 0],
              [-0.0858,0.0015,0.9963]])

transforms = np.array([np.eye(3), A, B])
s = incompetence_score(transforms)
print(f'Incompetence Score = {s:.3F}')

```

The output of the test code is shown in figure 6.1.

Incompetence Score = 25.912

Fig 6.1. Incompetence score

### 7.3.1. Hand Calculation

Given matrices:

$$A = \begin{bmatrix} 0.9998 & -0.0175 & 0 \\ 0.0175 & 0.9998 & 0 \\ 0 & 0 & 1 \end{bmatrix}, B = \begin{bmatrix} 0.9962 & -0.0174 & 0.0858 \\ 0.0175 & 0.9998 & 0 \\ -0.0858 & 0.0015 & 0.9963 \end{bmatrix}$$

Transition 1:  $I \rightarrow A$  (from Section 7.2)

$$\theta_1 = 1.146^\circ, \theta_1^2 = 1.313$$

Transition 2:  $A \rightarrow B$

Compute relative rotation:

$$R_{rel} = A^T \cdot B = \begin{bmatrix} 0.99630701 & 0.00009998 & 0.08578284 \\ 0.00006300 & 0.99990454 & -0.00150150 \\ -0.08580000 & 0.00150000 & 0.99630000 \end{bmatrix}$$

$$\text{tr}(R_{rel}) = 0.99630701 + 0.99990454 + 0.99630000 = 2.99251155$$

$$\cos \theta_2 = \frac{2.99251155 - 1}{2} = 0.996255775$$

$$\theta_2 = \arccos(0.996255775) = 4.960^\circ, \theta_2^2 = 24.602$$

Total incompetence score:

$$s = \theta_1^2 + \theta_2^2 = 1.313 + 24.599 = 25.915$$

the following table is shown result comparison between Hand and Python calculation.

| Method           | Score  |
|------------------|--------|
| Hand Calculation | 25.915 |
| Python Function  | 25.912 |

The two results are in excellent agreement, with a negligible difference of 0.003. This minor discrepancy arises from rounding errors in the hand calculation (using limited decimal places) versus the full double-precision arithmetic used by NumPy in the Python function.